READING BRIDGE

From copy-number profiles to ancestor messages

A Python-first explanation of the full L_LOH / two-WGD implementation.

1 Define the model

What histories are legal, and what do their costs mean?

2 Search only what matters

Generate compatible transitions lazily. Use proven bounds. Preserve every local tie.

3 Follow the implementation

Python schedules the work. C++ executes kernels. OpenFST and Boost supply standard algorithms.

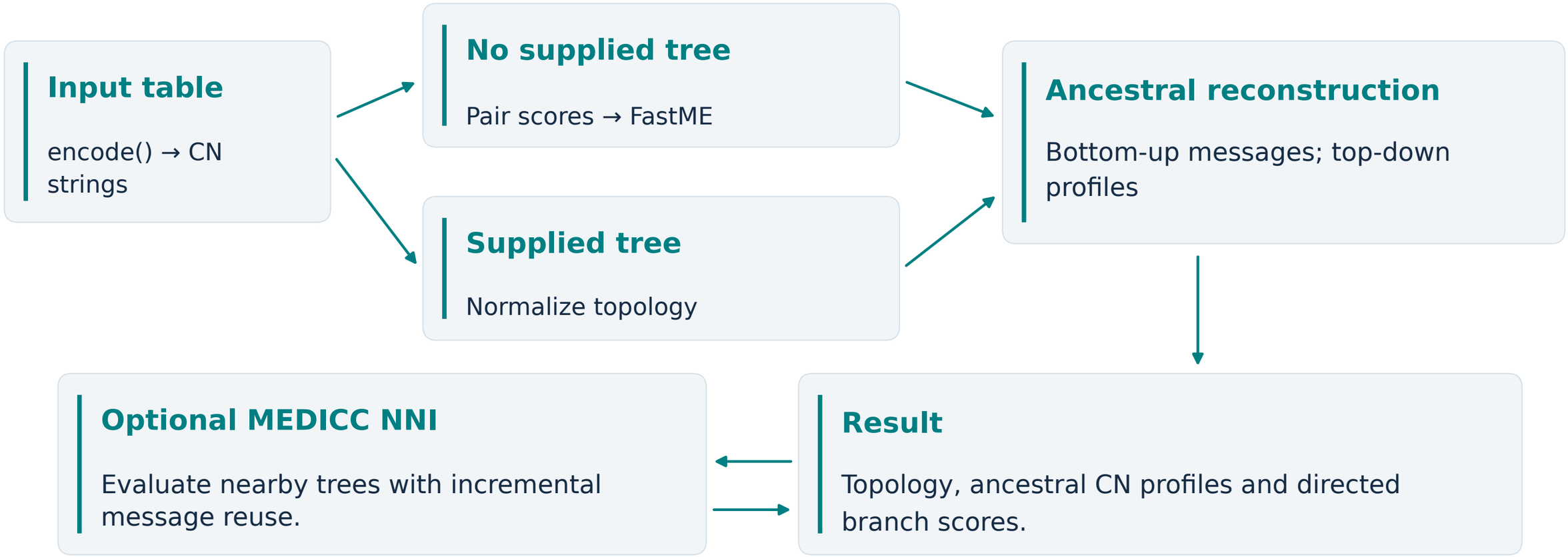
By the end, you should be able to explain:

```
score(x, y)     join(left, right)     resolve(parent, child)
```

You can understand the algorithm before reading pointer or template syntax.

Where the new backend sits in MEDICC

The numerical kernels are C++; the application and tree orchestration remain Python.



Code: medicc/full_fst/core.py::reconstruct, distance_matrix, infer_tree; cli.py::main

An FST is a graph of allowed transformations

FST = finite-state transducer. An arc carries an input symbol, output symbol and cost.

One path, shown across two bins:



Input 21 → output 10
One contiguous loss event: cost 1.

Min-plus arithmetic

Add costs along a path. Take the minimum over alternative paths. An impossible transformation has cost ∞.

Why state matters

Opening an event costs 1; continuing that unit loss costs 0. State remembers whether the event is open.

L_LOH allows the loss to reach zero within the same event; gains cannot resurrect a zero allele.

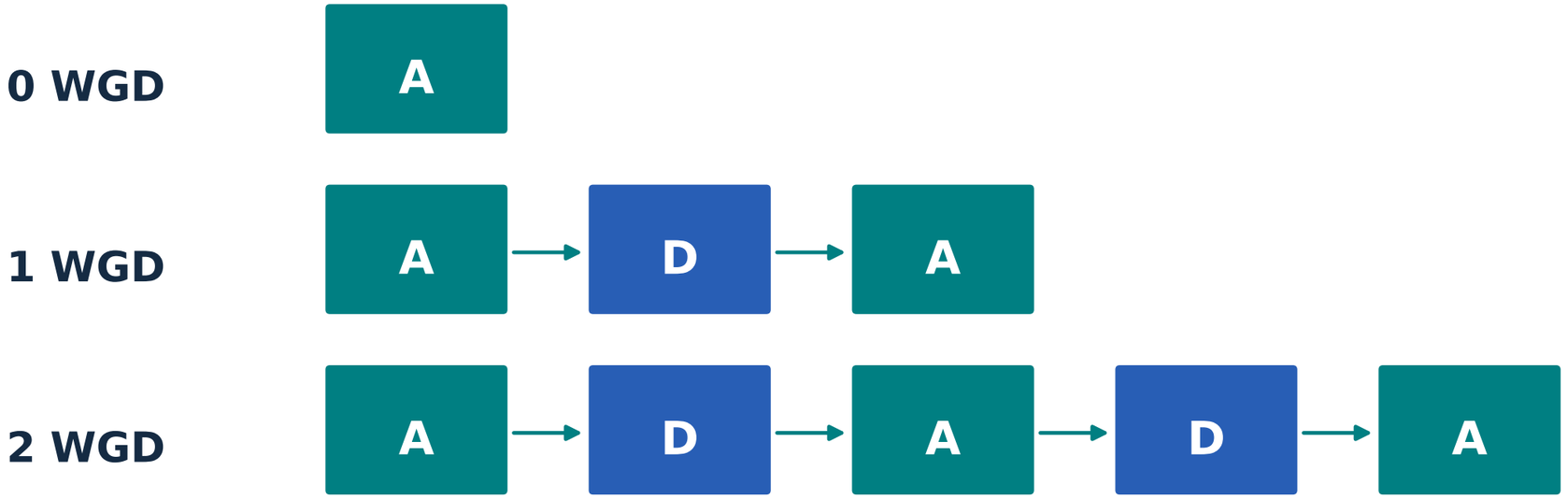
Code: medicc/factory.py::create_1step_del_fst; CN strings use symbols 0..8 and the separator X.

Full L_LOH is allowed around every doubling

A is the fused loss-plus-gain relation. D is a mandatory whole-genome doubling.

$$A = L_{\mathrm{LOH}}\, G$$

Read products left to right: the output of one stage feeds the next.



Choose the cheapest legal branch.

At most two WGDs per directed edge. Each costs 1 for the whole profile, not once per X block.

Code: ancestors/src/nfa_engine.hpp::branches; shared_engine.hpp::am_load (native paths under medicc/cext/full_fst/).

The large relation T is a definition, not an allocated graph

Eagerly composing the full cascade creates products of primitive states before we know what is useful.

Conceptual relation

T = minimum over A, A D A, A D A D A.

A sibling join also couples two such histories to two child messages.

Actual representation

Load the small fused A table.

Keep a tuple of primitive states and child-message states. Generate transitions only when the search asks for them.

State space can still be large. Laziness avoids materializing unused combinations; it is not a polynomial-time guarantee.

Search the bounded product first; determinize the retained ancestor language afterwards.

Code: ancestors/src/nfa_engine.hpp::Side, search, branches; assets/ancestral_parts.txt stores the fused primitive A.

Three related optimization problems

T(p, c) denotes the least cost of an allowed directed history from parent p to child c.

Directed score

Fixed parent and fixed child.

Returns one number: T(p, c).

Pair score

Fixed observed profiles x and y.

Minimize over their shared ancestor z.

Message join

Each child may represent many profiles.

Return ALL locally optimal ancestor profiles.

$$K(x, y) = \min_{z} [T(z, x) + T(z, y)]$$

A scalar score may discard equal-cost alternatives. A message join must preserve them.

Code: full_fst/pair_scorer.py::Model.score; model.py::Model.join, Model.resolve

Factorize across X blocks, after fixing the WGD counts

For fixed left/right WGD counts, separators reset primitive event states and blocks can be scored independently.

$$K(x,y) = \min_{k_l,\,k_r \in \{0,1,2\}} \left[k_l + k_r + \sum_b C_b(k_l, k_r) \right]$$

C_b is the exact event cost for one X-delimited chromosome/allele block, excluding the WGD charge.

One global choice

Evaluate up to 9 pairs (k_l, k_r). Sum block costs under that same pair. Charge k_l + k_r once.

Reuse exact block scores

Order branches by lower bounds. Cache completed exact costs. A cutoff failure is not cached as an exact answer.

Verified example: 1X1 → 2X2 costs 1 by one WGD, although it contains two blocks.

Code: direct_pair/src/pairwise.cpp::factorized::Chunk, factorized::score, am_pair_score

A message is a compressed set of profiles and costs

NFA = nondeterministic finite automaton; DFA = deterministic finite automaton. Both can encode weighted languages.

Python mental model

{'1': 1, '2': 1}

Two accepted ancestor profiles, each with subtree cost 1.

Native representation

A fixed-length, layered DAG.
Each state has ten symbol slots.
Each message stores a cost offset, arc weights and final weights.

$$M(w) = \text{offset} + \sum \text{arc costs} + \text{final cost}$$

Locally pruned output stores its common optimum in the offset; accepted-path residual costs are zero.

NFA: several same-symbol choices. DFA: at most one outgoing transition per symbol.

Code: full_fst/message.py; common/src/automata_types.hpp::Node, DFA

What join(left, right) computes

The minimization ranges over profiles still accepted by the two retained child messages.

$$F_v(z) = \min_{x,y} [T(z,x) + M_l(x) + T(z,y) + M_r(y)]$$

$$m_v = \min_z F_v(z)$$

Keep all local ties

M_v(z) = m_v when F_v(z) = m_v.
All other profiles receive cost ∞.

Leaf initialization

M_leaf(observed profile) = 0.
Every other word has cost ∞.

This is local prune_weight=0. Nonminimal profiles are discarded before the next tree join.

Code: full_fst/model.py::_join; ancestors/src/nfa_engine.hpp::am_nfa_join, search

A real one-bin example: keep both 1 and 2

These values were checked with the current Model and PairScorer implementations.

candidate z	T(z, 1)	T(z, 2)	join total
1	0	1	1
2	1	0	1

The join returns

{'1': 1, '2': 1}

One representative would lose information needed later.

Parent profile is 2

resolve("2", message) → ("2", 1)

Parent profile is 1

resolve("1", message) → ("1", 1)

Verified with Model.string("1"), Model.string("2"), Model.join and Model.resolve; model.py / message.py.

Lazy composition generates one column at a time

A joint state couples the left history, right history and their child-message states.

Current tuple

Primitive states + child states

→

Choose ancestor symbol

The same symbol feeds both sides.

→

Compatible successors

Match intermediate labels; sum costs.

Advance one bin

Each surviving successor advances both child messages consistently.

Keep state IDs, not a giant arc cache

Store reached states. Regenerate transitions for later backward/tight-path passes.

This bounded, model-specific composition engine remains custom C++.

Code: ancestors/src/nfa_engine.hpp::Side::prepare, Side::edges, StateKey, SearchState, search

A* uses certified lower and upper bounds

g is the cost already paid; h is an admissible lower bound on the remaining cost; U is a feasible full cost.

$$f(s) = g(s) + h(s) \qquad \text{discard only if} \quad f(s) > U$$

Where U comes from

Choose feasible child profiles. Score their pair exactly and add their stored subtree costs. Improve U whenever a cheaper complete history is found.

Where h comes from

Relaxed suffix dynamic programs and chromosome bounds. Add costs only when they are disjoint; use max for overlapping alternatives.

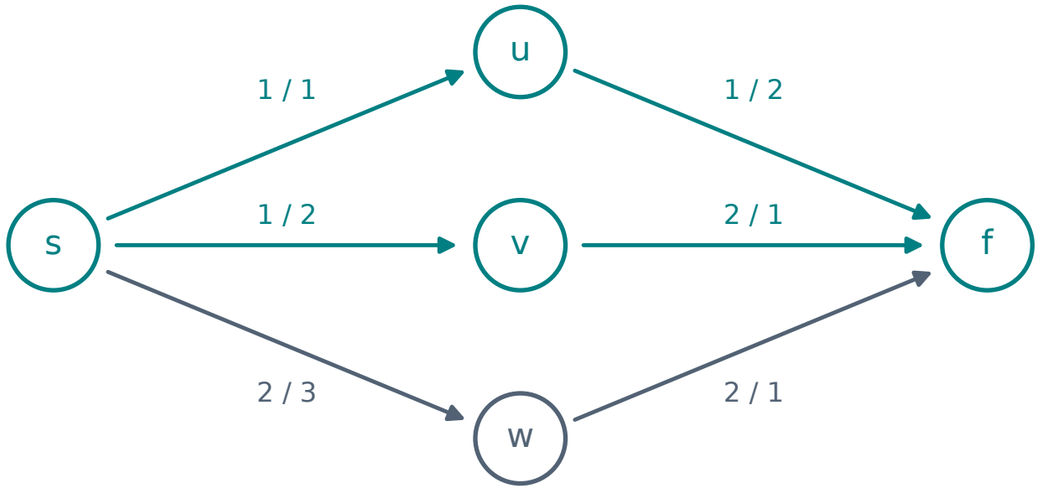
Illustration: U = 8. g = 5, h = 4 → discard. g = 5, h = 3 → keep the possible tie.

For all-tie joins, equality must survive. This is not a beam search or an empirical cutoff.

Code: model.py::_join; nfa_engine.hpp::Side, search; common/src/boost_search.hpp::run

Recover all optimal paths with a backward pass

Finding the optimum value is only the first step. The retained language must include every tying word.



Optimal cost C* = 3

11 costs 3; 12 costs 3. Keep both.
22 costs 4. Discard it.

Compute exact suffix costs on the explored region in reverse layer order.

$$\text{tight arc}: \quad g(s) + w(s,t) + \text{tail}(t) = C^*$$

Preserve the union of tight paths; a single predecessor traceback is insufficient here.

Code: ancestors/src/nfa_engine.hpp::search — layers, tail, kept; diagram is an illustrative graph.

OpenFST operates on the surviving language

The expensive history product is filtered before the standard language operations are applied.

Tight NFA

Only locally optimal ancestor words. Store their common cost in an offset.

→

OpenFST determinize

Merge alternative same-word paths into a deterministic acceptor.

→

Integer DAG minimizer

Normalize residual costs; merge equivalent suffix states.

Across WGD branches:

Keep every branch attaining the best join cost → OpenFST weighted union → epsilon removal → determinization → custom acyclic minimization.

OpenFST label 0 is epsilon. MEDICC symbols 0..9 are mapped to OpenFST labels 1..10.

Code: nfa_engine.hpp::determinize; shared_engine.hpp::unite; common/src/openfst_language.cpp; budget_2m_core.hpp::minimize

resolve(parent, message) chooses one optimal word

Here the parent profile is fixed and a single representative is sufficient.

$$c^* \in \operatorname{argmin}_c [T(p, c) + M(c)]$$

Search state

One primitive cascade plus the child DFA state. The next parent symbol is already known. Explore the 0-, 1- and 2-WGD branches.

Traceback

Boost A* runs over generated edges. Predecessor state IDs and emitted symbols reconstruct one minimum-cost child string.

For the retained message {1: 1, 2: 1} and parent 2, choosing child 2 gives total conditional cost 1.

This search does not construct or determinize the full transducer T.

Code: full_fst/model.py::_resolve; ancestors/src/ancestors.cpp::direct_trace::search, am_direct_resolve

Read the two passes as ordinary Python

This is pseudocode for the scheduling logic; real messages remain compressed native objects.

Bottom-up: retain possibilities

```
for leaf in leaves:
    M[leaf] = model.string(observed[leaf])

for v in bottom_up_order:
    M[v] = model.join(M[left[v]], M[right[v]])
```

Top-down: choose profiles

```
profile[normal] = observed[normal]

for v in top_down_order:
    profile[v], cost = model.resolve(
        profile[parent[v]], M[v])
```

Leaf profiles remain observed. Directed branch scores use the selected endpoint profiles.

The ctypes boundary changes representation, not this Python-level control flow.

Code: full_fst/core.py::reconstruct; scheduler.py::reconstruct; model.py::string, join, resolve

NNI reuses messages whose subtrees did not change

MEDICC proposes neighboring topologies; the backend updates only the affected reconstruction work.

Bottom-up cache

Copy the cache mapping, sharing immutable message owners.

Mark the affected path toward the normal root dirty. Recompute those joins.

Top-down cache

Reuse a previous choice only if BOTH hold:

1. The complete upper message is unchanged.
2. The resolved parent profile is unchanged.

An unchanged subtree can still need a different chosen ancestor when its parent changes.

The explicit FULL_TWO sentinel selects this backend; no runtime monkeypatch is required.

Code: medicc/core.py::nni_mode; full_fst/ancestors.py::reconstruct, Cache, CachedMessage

PERFORMANCE

Reduce repeated work without changing the model

These optimizations have distinct scopes; none is a search-state cap.

Pair shortening

Collapse identical adjacent column pairs for pair/branch scoring. Keep X boundaries.

Ancestral messages retain the full bin grid.

Parallel scheduling

Ready tree nodes can run concurrently. Each worker owns its mutable Model.

Completed messages are read-only inputs.

Bounded work sharing

When branch parallelism is enabled, ExecutionBudget shares worker permits between tree tasks and WGD helpers.

Cache exact costs and reuse immutable messages; do not confuse a cost bound with an arbitrary cutoff.

Code: full_fst/shortening.py; model.py::shorten, ExecutionBudget; scheduler.py; direct_pair/src/pairwise.cpp

Read a native handle as an owned resource

Python wrappers establish lifetimes. Raw addresses stay inside the binding and kernel layers.

Model owns AM

am_load allocates it.
Model.close → am_free.

Message owns DFA

join/string/import allocate it.
Message.close → am_drop.

Inputs are borrowed

join and resolve read input messages; they do not free them.

Indices are easier than pointers

Search nodes store predecessor IDs. A vector can move its elements when growing; reacquire nodes[id].

Python keeps the owners alive

A Message retains its Model. CachedMessage retains a Message. Do not close a message still shared by the NNI cache.

Code: full_fst/model.py; message.py; pair_scorer.py; _bindings.py; native C ABI comments

What belongs to libraries, and what belongs to MEDICC

The boundary follows measured performance and the specialized model semantics.

OpenFST

Ancestral determinization
Weighted union
Epsilon removal

Small value-type adapter in openfst_language.cpp.

Boost.Graph

Ancestral forward A*
Conditional traceback search

The adapter exposes generated edges, distances and stable tie ordering.

MEDICC / custom C++

Model primitives and bounds
Bounded lazy composition
Tight-path extraction
Integer DAG minimization
Specialized state indexing
Fixed-profile pair search

New kernels: C++17 via ctypes + C ABI. Legacy FSTlib and acyclic pruning: Cython.

Code: common/src/openfst_language.cpp, boost_search.hpp; ancestors/src/nfa_engine.hpp; direct_pair/src/pairwise.cpp

What “exact” means here

Keep numerical-kernel exactness separate from the approximation implied by local message pruning.

Guaranteed by the intended semantics

Exact pair/edge scores for the fixed model.

All locally minimum ancestor words retained.

Exact conditional choice within a retained message.

CN 0..8 and ≤2 WGDs per edge.

Not a global-tree optimality claim

A locally nonminimal ancestor is removed. It could have helped a later parent connection.

NNI searches nearby topologies; it does not enumerate every tree.

Evidence: 52,488 pair-oracle comparisons; 240 message cases; weighted-message and NNI/cache checks; 54 before/after workflow runs for the readability refactor.

These checks cover the recorded model and workloads. The complete 900-dataset arrays were not rerun.

Sources: tests/check_pairs.py, check_oracle.py, check_api.py, check_bindings.py; README.md validation records.

NOW OPEN THE CODE

One reading route, with concrete questions

Use the public Python entry first. Follow one join into C++ before reading the generic graph adapter.

1. **medicc2 → medicc/full_fst/cli.py::main**
   How do input options reach the public API?
2. **medicc/full_fst/core.py::reconstruct**
   Which topology route and tree pass are selected?
3. **medicc/full_fst/scheduler.py → model.py → _bindings.py**
   Who schedules, who owns memory, and which native function runs?
4. **ancestors/src/nfa_engine.hpp::search**
   Where are successors generated, bounded and retained as tight paths?
5. **common/src/openfst_language.cpp + boost_search.hpp**
   Which standard algorithms do the adapters call?
6. **ancestors/src/ancestors.cpp; medicc/full_fst/ancestors.py**
   How does one profile resolve, and what may NNI reuse?

Skip src/generated/ tables and the compatibility aliases pipeline.py / integration.py on the first pass.

Paths on this page are relative to the repository. Native paths start at medicc/cext/full_fst/.